Lab 6 report

Section 1: Edge Impulse Results

Public Edge Impulse project link: https://studio.edgeimpulse.com/public/996854/live

Part 1 questions:

Q1: Raw inputs: State the raw inputs used in the Edge Impulse project.

A: Raw inputs are 3-axis accelerometer time-series signals: accX, accY, and accZ. The impulse uses a 2000 ms window, 220 ms stride, and 100 Hz sampling frequency, with zero-padding enabled.

Q2: NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

A: Feature type: Spectral features. Number of NN input features: 33 (shown as the NN input layer size). Example feature names used by the classifier include: accX RMS, accX Peak 1 Height, accY Peak 3 Height, accX Peak 1 Freq, and accY RMS.

Q3: NN classifier outputs: State the outputs of the NN classifier.

A: The NN classifier outputs two class probabilities: nominal and off.

Q4: Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

A: Feature type: Spectral features. Number of selected anomaly features: 4. Selected feature names: accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 - 5.0 Hz, and accZ Spectral Power 2.0 - 5.0 Hz.

Q5:

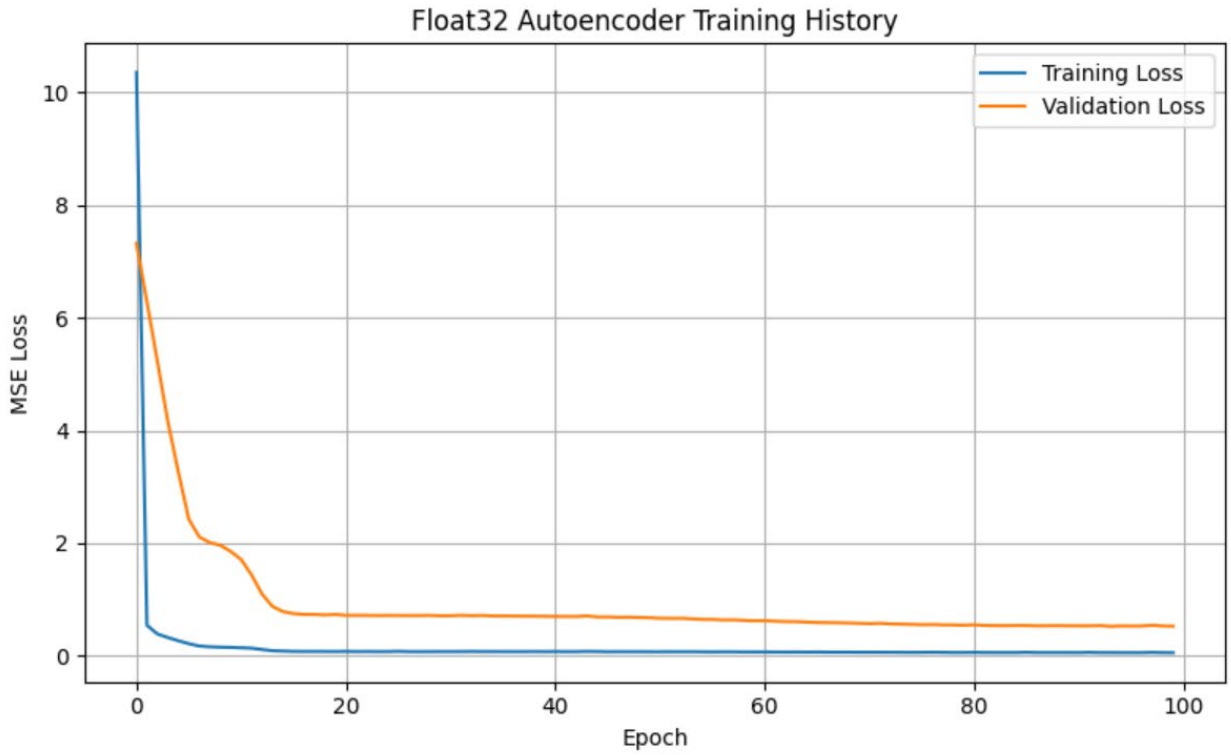
```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 25 ms., Classification: 1 ms., Anomaly: 1 ms.):
    nominal: 0.34375
    off: 0.65625
    anomaly score: 78.615
```

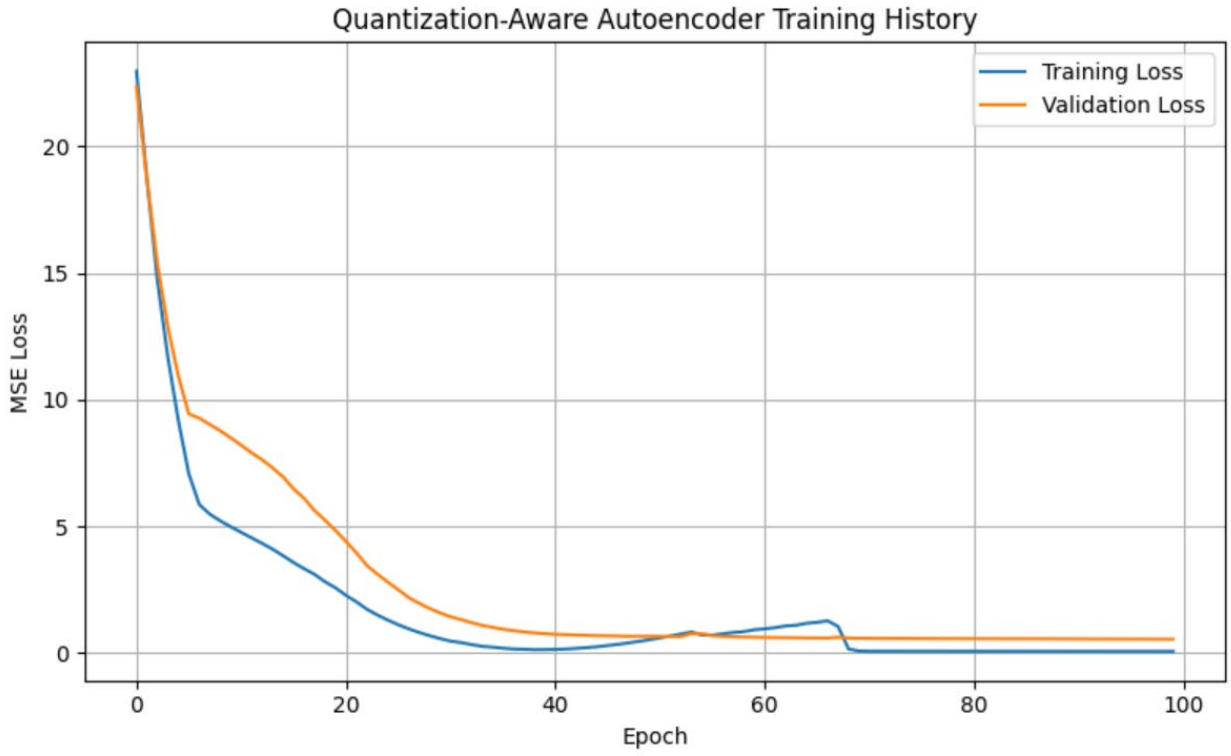
Q6: Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

A: Observed results are strong (high validation accuracy and clear class separation), but outputs should not be blindly trusted in all conditions. Reliability is best when deployment conditions match training conditions (sensor orientation/placement, motion patterns, environment, and sampling behavior). Reliability drops under domain shift, noisy signals, unseen activities, or hardware/runtime differences. In practice, use class probabilities together with the anomaly score and sanity-check behavior on real deployment data.

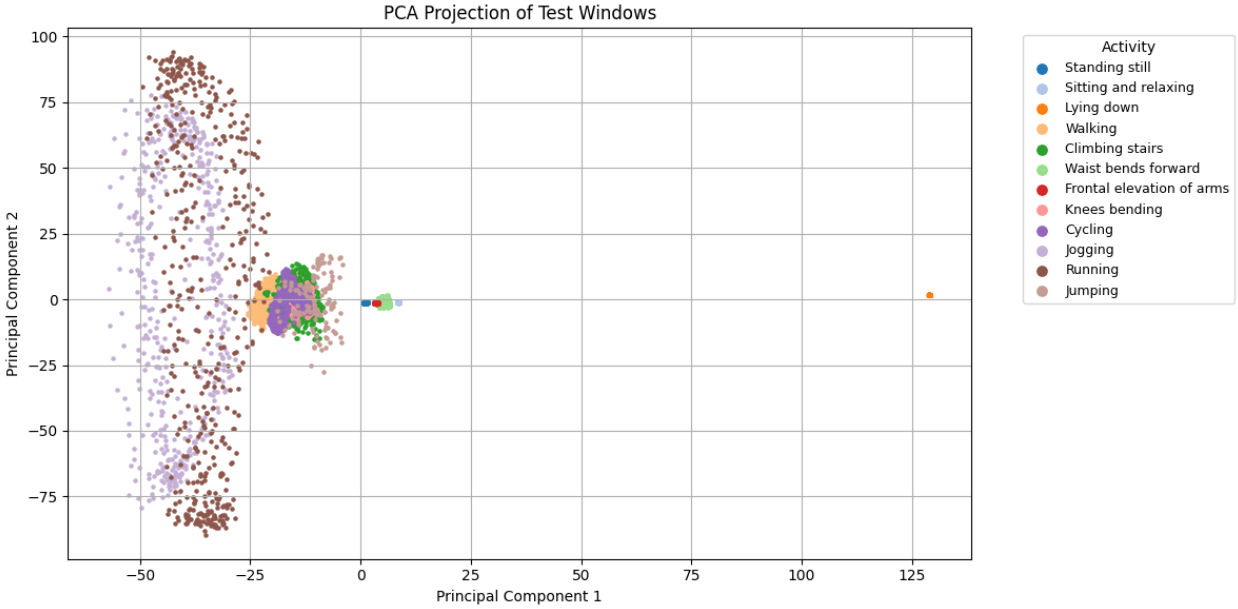
Section 2: Autoencoder Model Results

• Training and validation loss curves

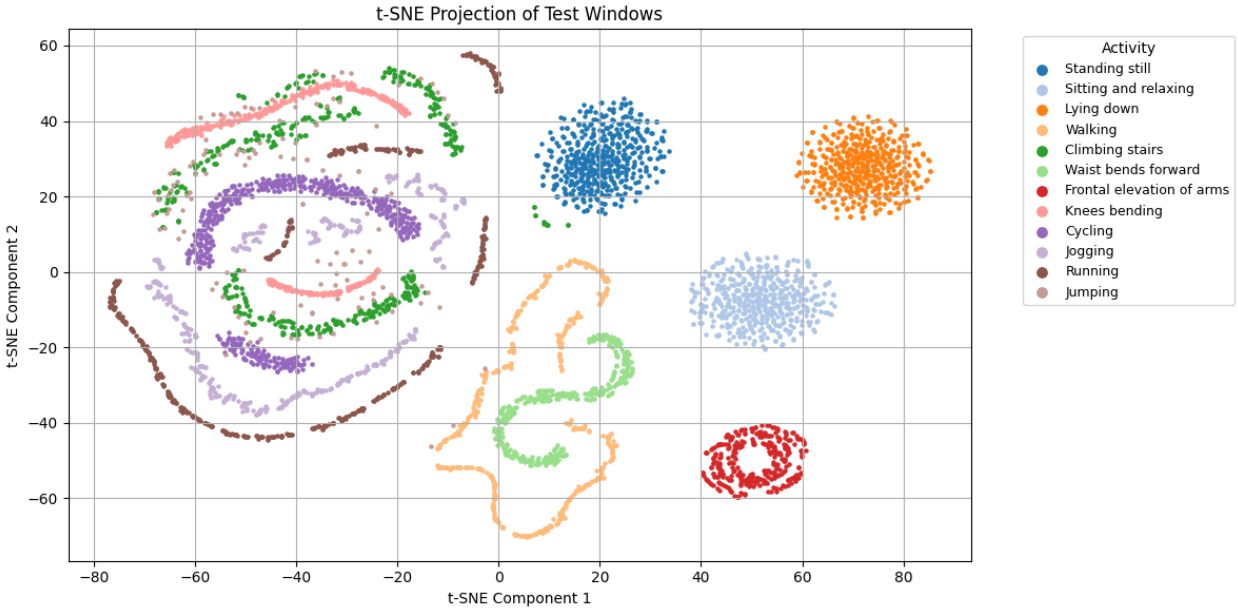




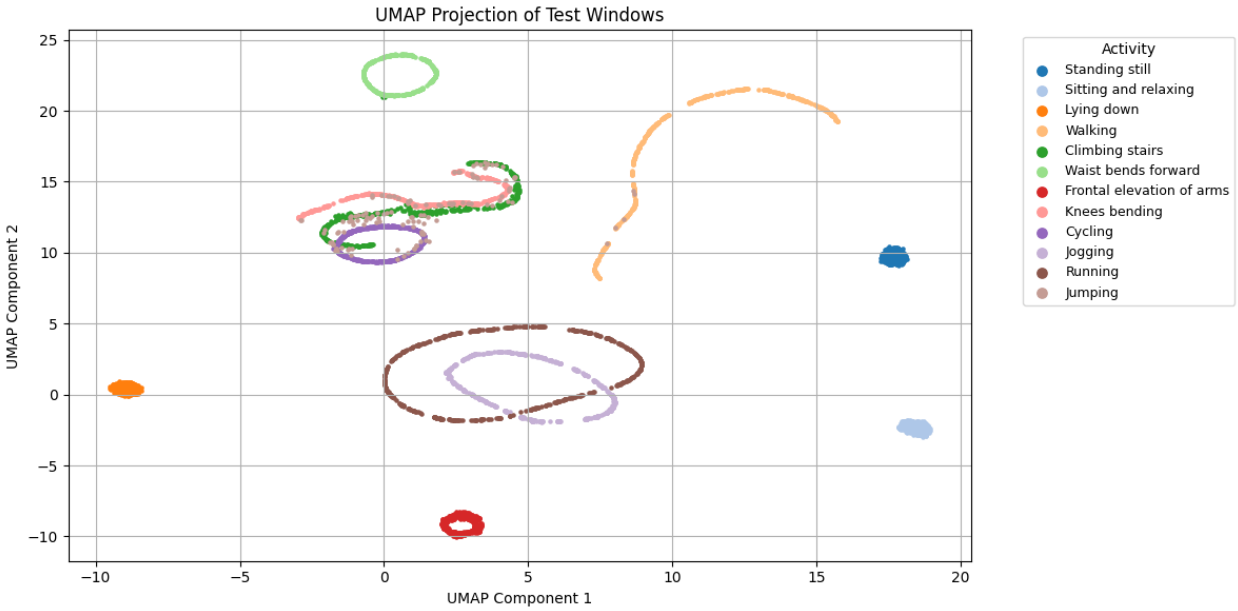
- PCA visualization



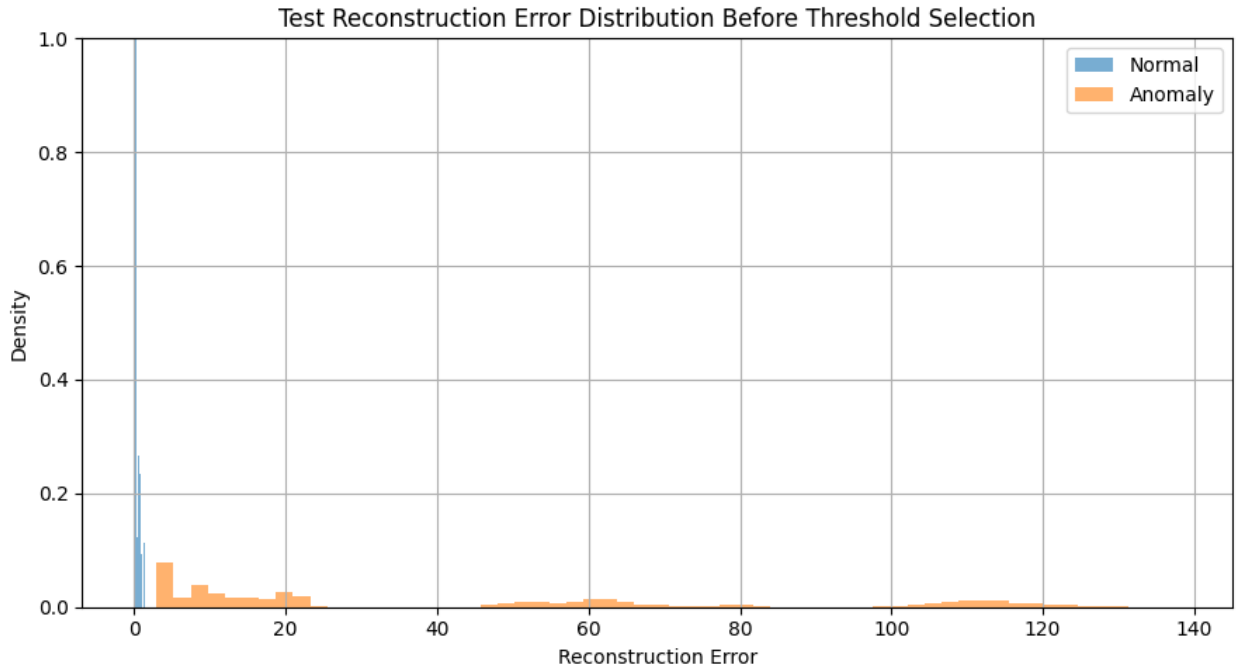
• t-SNE visualization

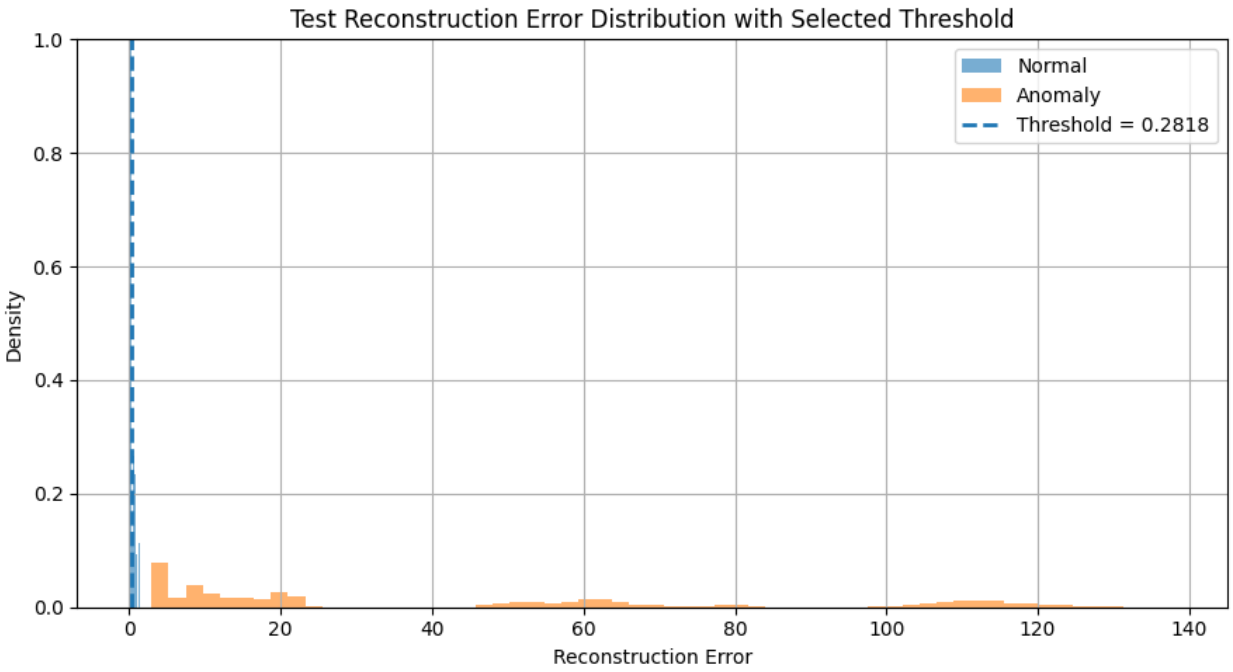


• UMAP visualization

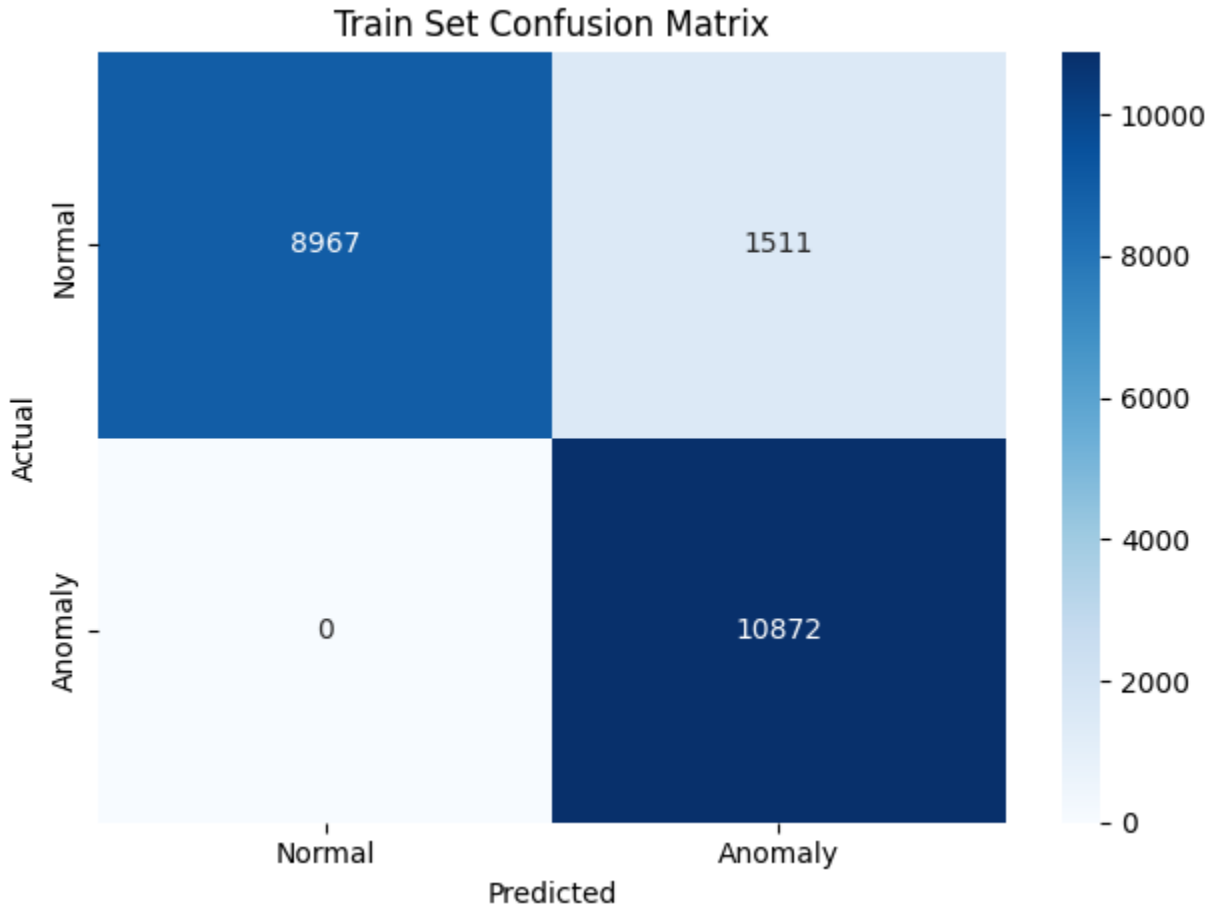


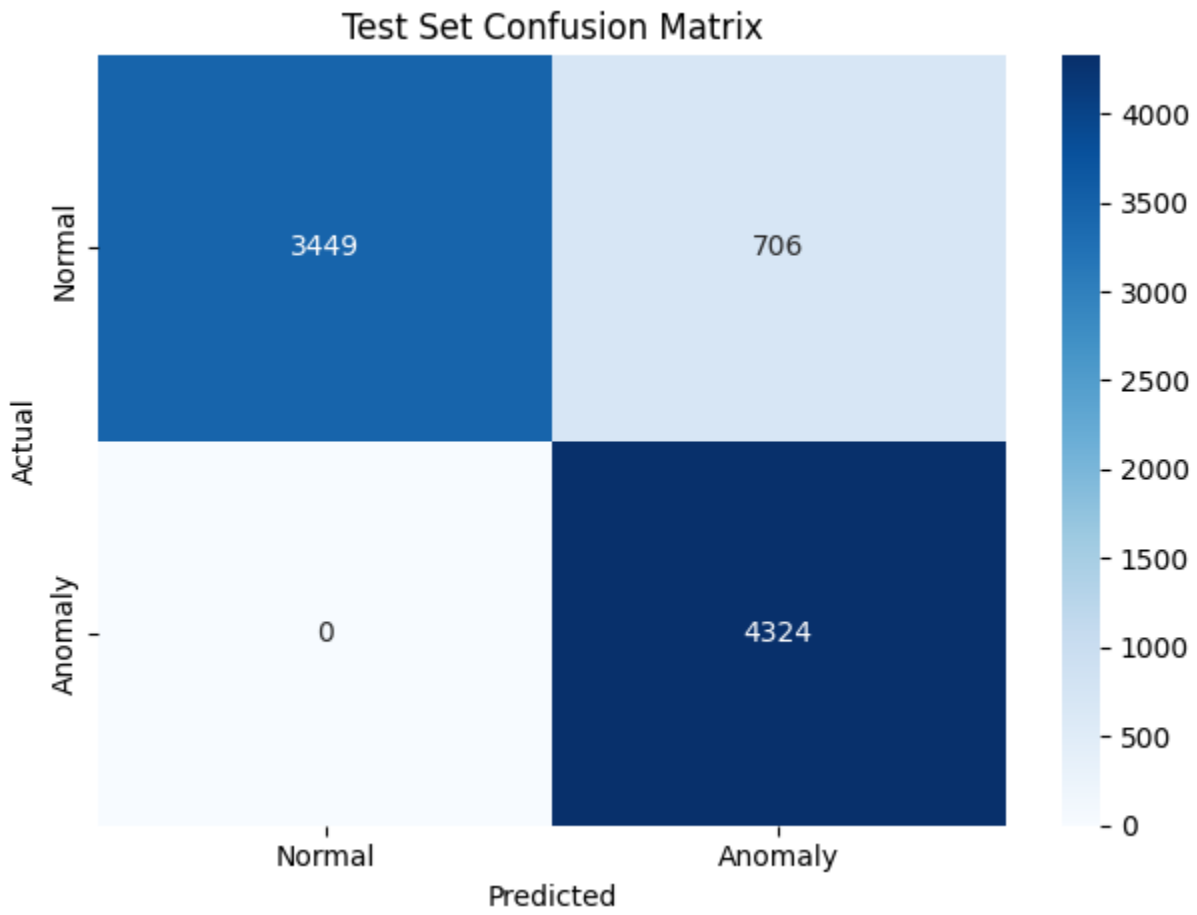
• Reconstruction error distribution and Selected anomaly threshold





• Confusion matrix





• Classification report or evaluation metrics

```
Train normal       mean=0.103488, stderr=0.001742, min=0.008221, max=1.124084
Train anomaly      mean=42.173885, stderr=0.369830, min=2.448245, max=136.356842
Test normal        mean=0.149351, stderr=0.004233, min=0.008420, max=1.419557
Test anomaly       mean=42.524750, stderr=0.619171, min=2.951014, max=138.136902
```

```
Train set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      0.86      0.92     10478
 Anomaly (1)       0.88      1.00      0.94     10872

    accuracy                           0.93     21350
   macro avg       0.94      0.93      0.93     21350
weighted avg       0.94      0.93      0.93     21350

Test set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      0.83      0.91      4155
 Anomaly (1)       0.86      1.00      0.92      4324

    accuracy                           0.92      8479
   macro avg       0.93      0.92      0.92      8479
weighted avg       0.93      0.92      0.92      8479
```

• Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```
Reconstruction error: 4.804316
Result: Anomaly detected
Reconstruction error: 4.739975
Result: Anomaly detected
Reconstruction error: 4.667830
Result: Anomaly detected
```

```
Reconstruction error: 0.214173
Result: Normal activity
Reconstruction error: 0.214137
Result: Normal activity
Reconstruction error: 0.214001
Result: Normal activity
```

Section 3: Discussion Questions

Q1: What threshold did you choose for anomaly detection, and why?

A: I used threshold = mean(train_normal_errors) + std(train_normal_errors), which in my run is about 0.282. This rule sets the cutoff just above typical normal reconstruction error, so windows that are unusually hard for the autoencoder to reconstruct are flagged as anomalies.

Q2: How does reconstruction error help distinguish normal and anomalous activity?

A: The autoencoder is trained only on normal windows, so it learns a compact representation of normal motion patterns and reconstructs them with low error. Anomalous patterns are out-of-distribution relative to that learned manifold, so reconstruction error is much higher. The error therefore acts as an anomaly score.

Q3: What information from the Python notebook must be preserved when deploying the model to Arduino?

A: You must preserve: (1) the exact model bytes (g_model) and model length (g_model_len) in autoencoder_model.cc, (2) the selected anomaly threshold (~0.282 for this run), (3) the same input shape/window definition (100 samples x 3 axes -> 300 features), and (4) the same quantization assumptions used by the INT8 model.

Q4: Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

A: The model was trained on a specific feature distribution. If Arduino uses different window length, axis order, scaling, sampling behavior, or flattening order, the input distribution shifts and

reconstruction error becomes unreliable. Matching preprocessing ensures the deployed model sees inputs in the same format it learned during training.

Q5: What are the main limitations of this anomaly detection method when deployed on a microcontroller?

A: Main limitations: fixed threshold may drift with sensor placement/user/context changes; unseen but valid normal activities can be falsely flagged; anomalous events similar to normal may be missed; tiny model capacity and INT8 quantization can reduce fidelity; and no online adaptation means performance can degrade over time without retraining.

AI disclosure:

Codex is used for code check and report writing style and logic check